

```

# -*- coding: utf-8 -*-
"""
Created on Sat Mar 28 03:28:42 2026
By sider massinissa universite de cergy physique Numerique
@author: E14
"""

# Ce programme est une Simulation numérique de l'équation de Schrödinger dépendante
du temps en dimension 1
# par la méthode de Crank-Nicolson.
# Analyse de l'effet tunnel et comparaison des resultats avec les schémas d'Euler
implicite et explicite
#dans la premiere partie dans chaque programme il faut importer les bibliotheques
utile et qui ne consomme pas beaucoup de temps lors de l'exécution

# *****IMPORTATIONS DES
BIBLIOTHEQUES*****

# numpy : calcul scientifique et manipulation de tableaux numériques
import numpy as np
# matplotlib : visualisation graphique des résultats
import matplotlib.pyplot as plt
# FuncAnimation : création d'animations dynamiques
from matplotlib.animation import FuncAnimation
# Axes3D : affichage des surfaces 3D (évolution spatio-temporelle)
#ATTENTION cette bibliothèque consomme plus d'espace sur l'ordinateur
from mpl_toolkits.mplot3d import Axes3D

# ----- CONSTANTES -----
# pour faciliter les calculs et gagner plus du temps on a pris les constantes
égales à 1
# afin de simplifier l'expression numérique de l'équation de Schrödinger. Cette
normalisation ne modifie pas la physique du système,
# mais permet d'améliorer la stabilité et la lisibilité du schéma numérique.
hbar = 1.0
m = 1.0

# ----- DISCRETISATION -----
# le principe de discrétisation de l'intervalle espace temps pour faciliter la
numérisation du problème
Nx = 300      # Points spatiaux
# nombre 600 implique que nombre d'actualisation du système lors de calculs est de
600 fois
# chaque pas se calcule en deux fois
Nt = 600      # Pas de temps
# L : longueur du domaine spatial
L = 10.0
# dx= pas spatial
dx = L / Nx
# l'intervalle entre deux pas du temps successive
dt = 0.01
# Création du maillage spatial uniforme
# On remplace l'espace continu par un réseau de points quantifiés.
# Chaque point représente une position possible où la fonction d'onde est évaluée.
# Plus Nx est grand : 1- meilleure précision 2-coût numérique plus élevé
x = np.linspace(0, L, Nx)

# ----- POTENTIEL -----
# initialisation de potentiel
# on considère que le potentiel est égal à zéro

```

```

# donc l'onde (paquet d'onde ) se propage librement dans l'espace sans aucune force
exterieur
V = np.zeros(Nx)
# Définit la largeur de la barrière en nombre de points du maillage.Nx
# Physiquement : contrôle l'épaisseur spatiale de la région d'interaction.
# Note :ON PEUT VARIER ET JOUE SUR L'EPAISSEUR DE BARRIERE OU DE PARTAGER LE
POTENTIEL A DES REGIONS
# QUI A UNE EPAISSEUR DIFFERENTE EN AUGMENTANT LE NOMBRE DE DIVISIONS OU DE CREER DES
INTERVALLES
barrier_width = int(Nx/20)
# la valeur de l'énergie potentielle
# c'est la valeur qui détermine s'il y aura réflexion partielle ou l'effet Tunnel
avec l'énergie du paquet d'onde
barrier_height = 10
# création d'un potentiel centré (rectangulaire) permettant d'étudier la diffusion
quantique (REFLEXION/TRANSMISSION)
V[Nx//2 - barrier_width//2 : Nx//2 + barrier_width//2] = barrier_height

# ----- CONDITION INITIALE -----
# les conditions initiales fixent l'état quantique du système à l'instant initial
# garantit l'unicité de la solution de l'équation
# dans la physique numérique on peut définir les conditions initiales comme un
vecteur de départ
# de l'algorithme qui détermine la dynamique ultérieure du système
# x0 : position initiale du paquet d'onde
x0 = L/4
# sigma représente la largeur du paquet d'onde gaussienne
sigma = 0.3
# le paquet d'onde gaussienne est plus stable numériquement et dans la simulation
# et est plus proche de l'aspect expérimental car il décrit différents phénomènes
dans l'espace
# Donc il décrit une particule concentrée autour de x0 mais avec une certaine
incertitude.
# les avantages :
# permet d'observer les phénomènes physiques
# simple à manipuler
# respecte le principe d'incertitude
# k0 : nombre d'onde initial (lié à l'impulsion moyenne)
k0 = 5.0
# Construction d'un paquet d'onde gaussien modulé par une onde plane
# Cela représente une particule localisée avec une impulsion initiale
psi = np.exp(-(x - x0)**2/(2*sigma**2)) * np.exp(1j*k0*x)
# Normalisation pour assurer la conservation de la probabilité totale
psi /= np.sqrt(np.sum(np.abs(psi)**2)*dx) # Normalisation correcte

# ----- MATRICES CRANK-NICOLSON -----
# alpha : coefficient issu de la discrétisation du Laplacien
alpha = 1j * dt * hbar / (2*m*dx**2)
# Matrices A et B du schéma implicite car il y a discrétisation dans l'espace et
dans le temps
# Le schéma CN est inconditionnellement stable et conserve la norme
A = np.zeros((Nx,Nx), dtype=complex)
B = np.zeros((Nx,Nx), dtype=complex)
#NOTE: de type complexe car à la base l'équation contient le type des nombres
complex
# Boucle pour remplir les matrices tridiagonales A et B
# Ces matrices correspondent à la discrétisation implicite (A) et explicite (B)

```

```

# de l'équation de Schrödinger dépendante du temps
for i in range(1, Nx-1):
# Partie tridiagonale de la matrice A
# A[i,i-1] et A[i,i+1] correspondent aux voisins dans la discrétisation du
Laplacien
#Le signe négatif provient de la discrétisation implicit
# Ces lignes définissent les éléments de la matrice qui agit sur la fonction d'onde
à l'instant suivant.
    A[i,i-1] = -alpha
# La daigonale principale de la matrice
    A[i,i] = 1 + 2*alpha + 1j*dt*V[i]/(2*hbar) # ici 1 + 2*alpha provient de la
discrétisation de la dérivée seconde spatiale (énergie cinétique).
#Le terme 1j*dt*V[i]/(2*hbar) intègre l'énergie potentielle locale au point $i$
# le terme si dessus Définit la diagonale supérieure. Elle représente la diffusion
vers la droite.
    A[i,i+1] = -alpha
# La matrice B : le present t
#Ces lignes définissent la matrice qui agit sur la fonction d'onde actuelle.
# Remarquez que les signes sont opposés à ceux de $A$.
#Diagonale inférieure pour l'état actuel.
    B[i,i-1] = alpha
# Diagonale principale. C'est l'inverse (conjugué complexe sur la partie
temporelle) de l'élément de la matrice $A$.
    B[i,i] = 1 - 2*alpha - 1j*dt*V[i]/(2*hbar)
# Diagonale supérieure pour l'état actuel.
    B[i,i+1] = alpha

# Conditions aux limites
# En physique numérique, quand on travaille sur une grille finie,
# dnc il faut dire au programme ce qui se passe aux extrémités
# (le premier point d'indice 0 et le dernier point d'indice -1).
#Fixent le bord gauche.
A[0,0] = A[-1,-1] = 1
#C'est un facteur de "saut". Au lieu d'enregistrer la fonction d'onde à chaque
itération (ce qui consommerait énormément de mémoire RAM), on ne va stocker les
données qu'une fois toutes les 5 itérations. Cela permet d'avoir une animation
fluide sans ralentir l'ordinateur] = 1
# Fixent le bord droit.
B[0,0] = B[-1,-1] = 1

# ----- STOCKAGE DES DONNÉES -----
#Cette partie est très importante car elle nous permet d'apprendre plus
# le stockage et gestion des données
# exploiter les donnée et faire animations ou presentations....
# pour commencer :
#n_skip = 5 : C'est un facteur de "saut".
#Au lieu d'enregistrer la fonction d'onde à chaque itération
#(ce qui consommerait énormément de mémoire RAM),
# on ne va stocker les données qu'une fois toutes les 5 itérations.
# Cela permet d'avoir une animation fluide sans ralentir l'ordinateur
# stocker 1 frame sur 5 pour animation
n_skip = 5
#C'est une liste vide destinée à recevoir les "instantanés" (snapshots) de la
fonction d'onde
# $\psi$ au cours du temps. C'est grâce à cette liste que vous pourrez créer une
vidéo ou un GIF de la particule qui se déplace.
psi_frames = []
# le Suivi de l'Énergie (Diagnostic Physique)
# Ces trois listes servent à vérifier la validité de votre simulation.

```

```

# En physique, l'énergie est une grandeur fondamentale
# pour les lignes suivantes c'est des listes pour le stockage
#
E_kin = [] # Pour stocker l'Énergie Cinétique ( $E_c$ ) à chaque pas de temps
E_pot = [] # Pour stocker l'Énergie Potentielle ( $E_p$ ), liée au potentiel  $V[i]$ .
E_tot = [] # Pour stocker l'Énergie Totale ( $E_{\text{tot}} = E_c + E_p$ ).
E_total_frames = [] # pour stocker l'énergie total
# peux se varier l'utilisation de cette liste
# NOTE : Dans un système fermé, l'énergie totale doit rester constante.
# Si vous tracez E_tot et que vous voyez une courbe qui monte ou qui descend
# cela signifie que votre pas de temps dt est trop grand
# ou que votre schéma numérique est instable.

# ----- ÉVOLUTION (Crank-Nicolson) -----
# cette etape est très important dans le programme
# c'est elle qui va faire tourner le moteur de calcul
# On lance la boucle qui va répéter le calcul pour chaque pas de temps, de 0 jusqu'à Nt.
# Dans cette boucle les calculs vont se répéter chaque pas de t de 0 jusqu'à Nt
for t in range(Nt):
# C'est le cœur de la méthode de Crank-Nicolson.
# A = est une matrice
# B @ psi) cette partie permet de multiplier la matrice B par la fonction d'onde
actuelle (le présent).
# np.linalg.solve(A, ...) résout le système linéaire pour trouver le nouveau psi
(le futur).
# C'est l'équivalent de faire  $\psi^{n+1} = A^{-1} \cdot B \cdot \psi^n$ 
psi = np.linalg.solve(A, B @ psi)

# Energie cinétique
# NOTE : Pour obtenir l'énergie, on utilise l'opérateur Hamiltonien  $\hat{H} = \hat{T} + \hat{V}$ 
# Calcule la dérivée seconde spatiale de  $\psi = d^2\psi/dx^2$ 
# par la méthode des différences finies. np.roll permet de décaler le tableau pour
accéder aux voisins  $\psi_{i+1}$  et  $\psi_{i-1}$ .
d2psi = (np.roll(psi, -1) - 2*psi + np.roll(psi, 1)) / dx**2
# Calcule la densité d'énergie cinétique. On utilise le conjugué complexe  $\psi^* \hat{T} \psi$ 
car l'énergie est donnée par l'espérance  $\langle \psi | \hat{T} | \psi \rangle$ .
T = np.real(np.conj(psi) * (-hbar**2/(2*m) * d2psi))

# Energie potentielle
# Calcule la densité d'énergie potentielle locale  $|\psi(x)|^2 V(x)$ . Comme pour
l'énergie cinétique,
# on prend la partie réelle pour éviter les résidus numériques imaginaires infimes.
V_local = np.real(np.conj(psi) * V * psi)
# Intégration et Stockage des Énergies
# On passe d'une densité d'énergie à une énergie totale en sommant sur tout
l'espace
# (ce qui revient à faire une intégrale  $\int \psi^* \hat{H} \psi \, dx$ ).
# Energie totale
E_kin.append(np.sum(np.real(T))*dx) # Somme l'énergie cinétique sur toute la
grille
E_pot.append(np.sum(V_local)*dx) # Somme l'énergie potentielle
E_tot.append(np.sum(T + V_local)*dx) # Somme les deux pour vérifier la
conservation de l'énergie totale

# Stocker pour animation
# Si le reste de la division du temps par n_skip est zéro
#(donc toutes les 5 itérations ici), on enregistre les données

```

```

if t % n_skip == 0:

    psi_frames.append(np.abs(psi)**2) # On stocke la densité de probabilité
    # $|\psi|^2$  (ce que l'on voit réellement lors d'une observation).
    E_total_frames.append(T + V_local) # On stocke la répartition de l'énergie
pour l'afficher plus tard.
# une etape de traitement finale
psi_frames = np.array(psi_frames)
# Convertit la liste en un tableau NumPy robuste
#pour faciliter le traçage des graphiques.
# organisation de l'energie totale
# Transforme la liste de snapshots d'énergie (remplie durant la boucle)
# en un bloc de données solide (un tableau 2D)
# multiplier par T LE TRANSPOSER
E_total_frames = np.array(E_total_frames).T # Nx x Nt_frames
# Création de l'échelle temporelle réelle
# psi_frames.shape[0] : Récupère le nombre total d'images (frames)
# que vous avez réellement enregistrées.
# np.arange(...) : Crée une suite de nombres entiers (0, 1, 2, 3...)
# correspondant à l'indice de chaque image.
# * dt * n_skip : Convertit ces indices en temps physique
# (en secondes, par exemple). Comme on n'a enregistré qu'une image
# toutes les n_skip itérations, il faut multiplier par n_skip
# pour que l'axe du temps du graphique soit exact par rapport à la réalité de la
simulation.
time_values = np.arange(psi_frames.shape[0]) * dt * n_skip

# ----- FIGURE 3D DU PAQUET D'ONDES -----
#Ce dernier bloc de code sert à générer une image statique finale
# (et non une animation) représentant l'historique complet de Notre simulation
#en trois dimensions. C'est l'outil de visualisation ultime
#pour comprendre la "vie" de votre particule.

# ce ligne permet de Crée la fenêtre graphique
# avec une taille confortable (10x6 pouces).
fig3d = plt.figure(figsize=(10,6))
#Ajoute un système d'axes à l'intérieur de la figure.
#Le code 111 signifie "une seule grille", et projection='3d'
# active l'espace tridimensionnel.
ax3d = fig3d.add_subplot(111, projection='3d')
# Comme pour le flux énergétique,
# on crée un "maillage".
# Imaginez un filet de pêche posé au sol
# les nœuds du filet correspondent à chaque couple (Position, Temps)
# où nous avons calculé une valeur.
X, Y = np.meshgrid(time_values, x)
# la On prend la densité de probabilité stockée pendant la simulation.
# On utilise .T (la transposée) pour que les dimensions de Z (hauteur)
# correspondent parfaitement aux dimensions de la grille X et Y.
Z = psi_frames.T
# Dessine la surface 3D
ax3d.plot_surface(X, Y, Z, cmap='viridis')
#NOTE Hauteur (Z) : Représente  $|\psi(x,t)|^2$ ,
#soit la probabilité de trouver la particule
# à un endroit  $x$  et un instant  $t$ .
# Couleur (cmap='viridis') : Utilise une palette de couleurs
# allant du bleu sombre (probabilité nulle) au jaune (probabilité maximale).
# Nomme les axes pour rendre le graphique compréhensible.
# L'axe Z est ici la densité de probabilité

```

```

ax3d.set_xlabel("Temps")
ax3d.set_ylabel("Position x")
ax3d.set_zlabel(" $|\psi(x,t)|^2$ ")
#Donne un titre explicite au graphique.
ax3d.set_title("Distribution 3D du paquet d'ondes")
# Affiche enfin la fenêtre à l'écran.
plt.show()

# ----- ANIMATION 2D DU PAQUET D'ONDES -----
#Ce dernier bloc de code est celui qui génère le résultat le plus concret
# une vidéo (GIF) montrant le mouvement de la particule.
# C'est ici que vous pouvez réellement observer la physique à l'œuvre,
# comme l'étalement du paquet d'ondes ou l'effet tunnel.
# CES LIGNES C'EST POUR LA CONFIGURATION DES FIGURES
# Prépare la fenêtre de tracé.
fig2, ax2 = plt.subplots(figsize=(8,5))
# Initialise le graphique avec la fonction d'onde au temps  $t=0$ .
# On stocke cet objet dans la variable line pour pouvoir modifier ses données
# plus tard sans tout redessiner.
line, = ax2.plot(x, psi_frames[0])
#Dessine le potentiel  $V(x)$  (en rouge et pointillés).
# On le normalise (division par son maximum) pour qu'il soit à la même échelle
# que la fonction d'onde, permettant de voir où se trouvent les barrières
# par rapport à la particule.
ax2.plot(x, V/np.max(V), 'r--', label='Potentiel normalisé')
#Fixe les limites verticales.
# C'est essentiel pour éviter que l'axe "saute" pendant l'animation.
#Fixe les limites de l'axe vertical (ordonnées).
#On prend la valeur maximale de la probabilité rencontrée dans toute la simulation
# et on ajoute 20% de marge (1.2).
ax2.set_ylim(0, 1.2*np.max(psi_frames))
#Ajoutent les étiquettes indispensables.
#L'axe Y représente la densité de probabilité
# $|\psi|^2$ , c'est-à-dire la chance de trouver
#la particule à un endroit précis.
ax2.set_xlabel('Position x')
ax2.set_ylabel('|\psi|^2')
ax2.set_title('Animation densité de probabilité')
#Affiche la légende (pour distinguer la courbe de la fonction d'onde
# du tracé du potentiel rouge en pointillés).
ax2.legend()
# La fonction de mise à jour (update_2D)
# Cette fonction reçoit un numéro d'image (frame).
def update_2D(frame):

    line.set_ydata(psi_frames[frame])
    return line,
#C'est une optimisation de performance.
# Au lieu de tout effacer et de tout redessiner (ce qui est lent),
# on change uniquement les coordonnées verticales de la ligne existante avec
# les données de la frame correspondante.
    line.set_ydata(psi_frames[frame])
# Renvoie l'objet modifié pour que Matplotlib
# sache quelle partie du graphique rafraîchir.
    return line,
# Crée l'objet animation en liant la figure fig2 à la fonction update_2D.
# interval=50 signifie qu'il y a 50 millisecondes entre chaque image
# lors de la prévisualisation.
ani2 = FuncAnimation(fig2, update_2D, frames=len(psi_frames), interval=50)

```

```

# Générer physiquement le fichier sur votre ordinateur.
ani2.save("effet_tunnel.gif", writer="pillow", fps=30)
# Affiche l'animation dans une fenêtre interactive sur votre ordinateur.
plt.show()

# ----- ANIMATION 3D FLUX ÉNERGÉTIQUE -----
#Ce dernier bloc est le jumeau du précédent,
# mais appliqué à la troisième dimension pour l'énergie.
# Il permet de voir comment le "paysage énergétique"
# se construit au fur et à mesure que le temps s'écoule.
# Voici l'explication ligne par ligne :
# On crée une figure et on force l'axe à être en 3D.
#Sans l'option projection="3d", les commandes de surface
# ne fonctionneraient pas.
fig3, ax3 = plt.subplots(figsize=(10,6), subplot_kw={"projection": "3d"})
#Contrairement à la 2D où l'on change juste les données d'une ligne, en 3D
# avec Matplotlib, il est souvent plus simple d'effacer et de redessiner

def update_3D(frame):
# On nettoie le graphique précédent pour dessiner
#la nouvelle surface mise à jour.
    ax3.clear()
#On extrait les données d'énergie de l'origine jusqu'à l'instant frame.
#Cela crée cet effet de "vague" qui avance dans le temps.
#On extrait les données d'énergie de l'origine jusqu'à l'instant frame.
# Cela crée cet effet de "vague" qui avance dans le temps
    Z = E_total_frames[:, :frame+1]
#
    T_mesh = time_values[:frame+1]
# On crée la grille (le sol) sur laquelle l'énergie sera projetée en hauteur.
# X correspond au temps et Y à la position.
    X, Y = np.meshgrid(T_mesh, x)
# On dessine la surface. Le style 'plasma' est très utilisé pour l'énergie
# car il va du bleu/noir (énergie basse) au jaune/blanc (énergie haute).
    surf = ax3.plot_surface(X, Y, Z, cmap='plasma')

    ax3.set_xlabel("Temps")
    ax3.set_ylabel("Position x") #La fonction de mise à jour dynamique (update_3D)
    ax3.set_zlabel("Énergie totale locale")
    ax3.set_zlim(0, np.max(E_total_frames))
    ax3.set_title("Flux énergétique 3D")
    return surf,
#
    ax3.clear()
# Efface l'image précédente. En 3D,
#c'est souvent nécessaire pour éviter que les surfaces des instants précédents
# ne se superposent et ne créent des artefacts visuels.
    Z = E_total_frames[:, :frame+1]
# Sélectionne les données d'énergie.
# On prend toute la colonne spatiale (:)
# mais seulement les données temporelles de l'origine jusqu'à l'instant actuel
# (:frame+1). Cela donne l'impression que la surface "pousse" vers l'avant
    T_mesh = time_values[:frame+1]
# Sélectionne les points de temps correspondants pour que l'axe du temps
# soit synchronisé avec les données de Z.
    X, Y = np.meshgrid(T_mesh, x)
# Crée une grille de coordonnées.
# X représente les points sur l'axe du temps et Y
#les points sur l'axe de position. C'est la base (le sol)

```

```

# sur laquelle la surface va reposer.
    surf = ax3.plot_surface(X, Y, Z, cmap='plasma')
# Dessine la surface en 3D.
# La hauteur est définie par Z (l'énergie).
# cmap='plasma' : Applique une carte de couleurs allant du bleu/noir
# pour les valeurs basses au jaune pour les valeurs hautes.
# C'est idéal pour visualiser les flux d'énergie.
    ax3.set_xlabel("Temps")
# Définissent les noms des axes. Notez que votre code affiche des caractères
# comme  $\Gamma$ , ce qui indique un petit problème d'encodage
# (il s'agit du mot "Énergie").
# Crucial. En fixant une limite verticale constante dès le début,
# vous empêchez la caméra 3D de changer d'échelle
# à chaque fois que la valeur maximale de l'énergie fluctue.
# Cela rend l'animation fluide et stable.
    ax3.set_ylabel("Position x")
    ax3.set_zlabel(" $\Gamma$ nergie totale locale")
    ax3.set_zlim(0, np.max(E_total_frames))
    ax3.set_title("Flux  $\Gamma$ nergie 3D")
    return surf,
ani3 = FuncAnimation(fig3, update_3D, frames=E_total_frames.shape[1], interval=50)
# On enregistre le résultat. Le rendu 3D étant plus gourmand en calcul,
# cette étape peut prendre un peu plus de temps que pour la 2D.
ani3.save("flux_energetique.gif", writer="pillow", fps=30)
# AFFICHER L'ANIMATION SUR L'ORDINATEUR
plt.show()

# ----- GRAPHIQUES ÉNERGIES -----
plt.figure(figsize=(8,5))
# CREATION ESPACE de travail
# et de l'affichage pour les graphes
plt.plot(np.arange(Nt)*dt, E_kin, label="Énergie cinétique")
# Affiche l'évolution de l'énergie cinétique.
# Vous devriez voir cette courbe fluctuer si la particule ralentit ou accélère
# (par exemple en s'approchant d'une barrière).
plt.plot(np.arange(Nt)*dt, E_pot, label="Énergie potentielle")
# Affiche l'énergie potentielle.
# Elle augmente lorsque la particule entre dans une zone où le potentiel  $\Phi$  est
# élevé
plt.plot(np.arange(Nt)*dt, E_tot, label="Énergie totale")
# les " lignes suivantes sont pour la documentation d'affichage
# Donnent un sens aux axes. L'axe des abscisses est converti en temps réel
# (secondes) en multipliant le numéro du pas de temps par dt
plt.xlabel("Temps")
plt.ylabel("Énergie")
plt.title("Énergies au cours du temps")

plt.legend()
#Permet de distinguer les trois types d'énergies grâce à une légende.
plt.show()
# Affiche l'animation dans une fenêtre interactive sur votre ordinateur.

# ----- COMPARAISON MÉTHODES -----
# NOTE PHYSIQUE : Fonction psi_euler_explicite
# C'est la méthode la plus directe,
# mais elle est instable pour l'équation de Schrödinger
# (la fonction d'onde finit par "exploser").

def psi_euler_explicite(psi0, V, dx, dt, Nt):

```

```

# cette methode est vraiment importante dans la programmation
# On crée une copie locale de la condition initiale
# pour éviter de modifier la variable d'origine.
psi = psi0.copy()
# On récupère le nombre de points sur la grille spatiale
Nx = len(psi)
# On lance la boucle temporelle pour avancer de $N_t$ pas de temps.
for _ in range(Nt):
# Utilise np.roll pour décaler les voisins et calculer la dérivée seconde spatiale
#  $\frac{\partial^2 \psi}{\partial x^2}$  (le Laplacien).
# C'est le cœur de l'énergie cinétique.
d2psi = (np.roll(psi, -1) - 2*psi + np.roll(psi, 1)) / dx**2
# Mise à jour de la fonction d'onde selon la formule :
#  $\psi_{t+dt} = \psi_t + \frac{dt}{i\hbar} \hat{H} \psi_t$ .
# On ajoute à la valeur actuelle la variation calculée
# à partir de l'énergie cinétique et du potentiel $V$
psi += -1j * dt / hbar * (-hbar**2/(2*m) * d2psi + V*psi)
# On impose que la particule ne puisse pas sortir des bords
# (Conditions de Dirichlet)
psi[0] = psi[-1] = 0
# Renvoie la densité de probabilité finale (le carré du module complexe).
return np.abs(psi)**2
# Contrairement à la méthode explicite,
# La méthode implicite calcule l'état futur
# en résolvant un système d'équations, ce qui la rend beaucoup plus stable
# numériquement.
# ON DEFINIT LA METHODE AVEC LES VARAIBLES QUI RENTRE DANS CONSTRUCTION DE LA
# SOLUTION
# La fonction d'onde ; le potentiel; pas spacial; pas temporaire; nombre de
# repetition
def psi_euler_implicite(psi0, V, dx, dt, Nt):
# Récupère le nombre de points de la grille spatiale à partir
# de la fonction d'onde initiale.
Nx = len(psi0)

alpha = 1j * dt * hbar / (2*m*dx**2)
# Calcule le coefficient sans dimension qui regroupe les paramètres physiques
# (temps, espace, masse). Le 1j indique que l'évolution est imaginaire,
# typique de l'équation de Schrödinger.
A = np.zeros((Nx,Nx), dtype=complex)
# Initialise une matrice carrée vide de taille
# $N_x \times N_x$. On précise dtype=complex car
# la fonction d'onde possède une phase.
for i in range(1, Nx-1):
# Démarre une boucle pour remplir les lignes de la matrice (en évitant les bords).
A[i,i-1] = -alpha
# Définit la dépendance du point actuel par rapport au voisin de gauche.
A[i,i] = 1 + 2*alpha + 1j*dt*V[i]/(2*hbar)
# Définit la diagonale principale.
# Elle contient l'identité ($1$),
# la diffusion cinétique ($2\alpha$) et l'influence du potentiel local ($V[i]$).
A[i,i+1] = -alpha
# Définit la dépendance par rapport au voisin de droite
A[0,0] = A[-1,-1] = 1
# Impose les conditions aux limites de Dirichlet.
# On force les coins de la matrice à $1$ pour que les valeurs
# aux extrémités restent isolées et gérables.
psi = psi0.copy()
# Crée une copie de travail de la fonction d'onde

```

```

# pour ne pas écraser les données initiales.
    for _ in range(Nt):
# Répète le calcul pour chaque pas de temps
# jusqu'à atteindre le temps final.
        psi = np.linalg.solve(A, psi)
# C'est la ligne la plus importante. Au lieu de multiplier, on résout le système  $A \psi^{n+1} = \psi^n$ . Cela revient mathématiquement à appliquer l'inverse de la
matrice  $A$  à chaque étape.
        psi[0] = psi[-1] = 0
    return np.abs(psi)**2
# Calcule le module au carré de la fonction d'onde complexe. En physique, cela
correspond à la densité de probabilité de présence de la particule (la probabilité
de la trouver à l'endroit  $x$ ).
#*****NOTE PHYSIQUE*****
# Pourquoi utiliser cette méthode plutôt que l'explicite ?
# Même si np.linalg.solve demande plus d'efforts de calcul à l'ordinateur,
# cette méthode est dite "inconditionnellement stable".
# Là où la méthode explicite pourrait "exploser" et donner des nombres infinis,
# la méthode implicite restera toujours cohérente. Avez-vous remarqué que
# la matrice  $A$  est presque la même que celle de Crank-Nicolson,
# mais que la matrice  $B$  (le côté droit de l'équation) a disparu ?
#*****
# Conditions initiales
# cette partie est responsable pour crée paquet d'onde gaussienne
# la premier ligne Définit une enveloppe gaussienne (la forme en cloche).
psi0 = np.exp(-(x - x0)**2/(2*sigma**2)) * np.exp(1j*k0*x)
# Ajoute une phase complexe qui donne une vitesse initiale
#(quantité de mouvement  $p = \hbar k_0$ ) à la particule
psi0 /= np.sqrt(np.sum(np.abs(psi0)**2)*dx)
#Normalisation. On s'assure que la probabilité totale de trouver la particule
# dans l'espace est exactement 1.
#

# Calcul propagation finale avec les trois méthodes
# On récupère le dernier résultat de Crank-Nicolson
#(déjà calculé précédemment).
psi_CN = psi_frames[-1]
# On lance le calcul avec la méthode Euler Explicite.
psi_EE = psi_euler_explicite(psi0, V, dx, dt, Nt)
# On lance le calcul avec la méthode Euler Implicite.
psi_EI = psi_euler_implicite(psi0, V, dx, dt, Nt)
# les lignes de cette partie est responsable pour faire les calculs
# ----- FIGURE DE COMPARAISON -----
# Ce bloc de code final permet de réaliser une analyse comparative.
# Il affiche les résultats de trois méthodes numériques différentes
# pour résoudre la même équation de Schrödinger,
# vous permettant de juger de leur précision et de leur stabilité
plt.figure(figsize=(8,5))
# Crée une fenêtre de graphique de taille standard
plt.plot(x, psi_CN, label="Crank-Nicolson")
#Affiche le résultat de la méthode que vous avez codée précédemment.
# C'est généralement la courbe de référence
# car elle est la plus stable et précise.
plt.plot(x, psi_EE, label="Euler explicite", linestyle="--")
# Affiche le résultat de la méthode d'Euler Explicite.
# Note physique : Cette méthode est souvent instable
# pour l'équation de Schrödinger (la norme de la fonction d'onde
# a tendance à exploser).
plt.plot(x, psi_EI, label="Euler implicite", linestyle=":")

```

```

# Affiche le résultat de la méthode d'Euler Implicite.
# Note physique : Elle est stable, mais elle a tendance
# à "amortir" (diffuser) la fonction d'onde plus que nécessaire.
# CETTE DERNIERE PARTIE EST DEDIEE POUR
# DOCUMENTATION DE GRAPHIQUE
# Documentation du graphique
# plt.xlabel("Position x") et plt.ylabel(" $|\psi(x,t)|^2$ ") :
# Identifient les axes. L'axe Y est la densité de probabilité de présence de la
particule.

plt.title(...) : Donne un titre clair pour indiquer qu'il s'agit d'une comparaison
de l'état final.

plt.legend() : Affiche la légende en haut ou dans un coin,
# ce qui est indispensable ici pour savoir quelle couleur correspond
# à quelle méthode (CN, EE ou EI).
plt.show() : Affiche le graphique final à l'écran.
plt.plot(x, V/np.max(V), 'k--', label='Potentiel normalisé')

plt.xlabel("Position x")
plt.ylabel(" $|\psi(x,t)|^2$ ")
plt.title("Comparaison des méthodes numériques (densité finale)")
plt.legend()
plt.show()
# Affiche l'animation dans une fenêtre interactive sur votre ordinateur.

# Import de la bibliothèque numpy pour le calcul scientifique
# dans cette partie on cherche à calculer et présenter l'erreur de chaque méthode

import numpy as np

# Import de matplotlib pour les graphiques présenter les graphes sur différents
figures
import matplotlib.pyplot as plt

# ----- PARAMÈTRES PHYSIQUES -----

hbar = 1.0 # Constante de Planck réduite (unités naturelles)
m = 1.0    # Masse de la particule

L = 10.0   # Taille du domaine spatial
Nx = 200   # Nombre de points en espace
Nt = 500   # Nombre de pas de temps

dx = L / Nx    # Pas spatial
dt = 0.001     # Pas de temps libre à choisir cette valeur

x = np.linspace(-L/2, L/2, Nx) # Grille spatiale centrée

# Potentiel libre (aucune interaction)
# cette partie ne contient aucune interaction avec la barrière de potentiel
V = np.zeros(Nx)

# ----- CONDITION INITIALE -----

```

```

x0 = -2.0 # Position initiale du paquet d'onde
k0 = 5.0 # Impulsion (nombre d'onde)
sigma = 0.5 # Largeur du paquet

# Paquet d'onde gaussien avec oscillation plane
psi0 = np.exp(-(x-x0)**2/(2*sigma**2)) * np.exp(1j*k0*x)

# Normalisation pour que la probabilité totale = 1
psi0 = psi0 / np.sqrt(np.sum(np.abs(psi0)**2)*dx)

# ----- FONCTION PRINCIPALE -----

def analyze_methods(psi0, V, dx, dt, Nt):

    # Listes pour stocker la norme au cours du temps on a déjà expliqué
    l'importance des intervalles
    # dans le stockage des informations (les calculs)
    norm_CN = []
    norm_LF = []
    norm_EE = []
    norm_EI = []

    Nx = len(psi0) # Taille du système

    # --- CRANK-NICOLSON ---
    psi = psi0.copy() # Copie de l'état initial pour éviter d'écrire la même code
    a chaque fois

    # Coefficient numérique complexe
    alpha = 1j * dt * hbar / (2*m*dx**2)

    # Matrices du schéma implicite
    # construire une matrice discrète dans l'espace de taille NX NX

    A = np.zeros((Nx, Nx), dtype=complex)
    B = np.zeros((Nx, Nx), dtype=complex)

    # Remplissage des matrices (différences finies)
    # Donner des valeurs pour commencer les calculs
    for i in range(1, Nx-1):
        A[i,i-1] = -alpha
        A[i,i] = 1 + 2*alpha + 1j*dt*V[i]/(2*hbar)
        A[i,i+1] = -alpha

        B[i,i-1] = alpha
        B[i,i] = 1 - 2*alpha - 1j*dt*V[i]/(2*hbar)
        B[i,i+1] = alpha

    # Conditions aux bords (fixes) conditions aux limites sont déjà présenter
    # dans les premières parties de programme
    A[0,0] = A[-1,-1] = 1
    B[0,0] = B[-1,-1] = 1

    # Boucle temporelle
    for _ in range(Nt):
        psi = np.linalg.solve(A, B @ psi) # Résolution système linéaire
        norm_CN.append(np.sum(np.abs(psi)**2)*dx) # Calcul norme

```

```
# --- EULER EXPLICITE ---
```

```
psi = psi0.copy()
```

```
for _ in range(Nt):
```

```
    # Laplacien (approximation seconde dérivée)
```

```
    d2psi = (np.roll(psi, -1) - 2*psi + np.roll(psi, 1)) / dx**2
```

```
    # Application du Hamiltonien
```

```
    Hpsi = -hbar**2/(2*m) * d2psi + V*psi
```

```
    # Mise à jour explicite
```

```
    psi = psi - 1j * dt / hbar * Hpsi
```

```
    norm_EE.append(np.sum(np.abs(psi)**2)*dx)
```

```
# --- EULER IMPLICITE ---
```

```
psi = psi0.copy()
```

```
for _ in range(Nt):
```

```
    psi = np.linalg.solve(A, psi) # Résolution implicite
```

```
    norm_EI.append(np.sum(np.abs(psi)**2)*dx)
```

```
# --- LEAP-FROG ---
```

```
psi_p = psi0.copy() # état précédent
```

```
# Initialisation (premier pas)
```

```
d2psi = (np.roll(psi_p, -1) - 2*psi_p + np.roll(psi_p, 1)) / dx**2
```

```
Hpsi = -hbar**2/(2*m) * d2psi + V*psi_p
```

```
psi_n = psi_p - 1j * dt / hbar * Hpsi
```

```
# Stockage des deux premières normes
```

```
norm_LF.append(np.sum(np.abs(psi_p)**2)*dx)
```

```
norm_LF.append(np.sum(np.abs(psi_n)**2)*dx)
```

```
# Boucle Leap-Frog
```

```
for _ in range(2, Nt):
```

```
    d2psi = (np.roll(psi_n, -1) - 2*psi_n + np.roll(psi_n, 1)) / dx**2
```

```
    Hpsi = -hbar**2/(2*m) * d2psi + V*psi_n
```

```
    # Schéma centré en temps
```

```
    psi_f = psi_p - 2j * dt / hbar * Hpsi
```

```
    # Décalage des états
```

```
    psi_p, psi_n = psi_n, psi_f
```

```
    norm_LF.append(np.sum(np.abs(psi_n)**2)*dx)
```

```
# Retour des résultats
```

```
return norm_CN, norm_LF[:Nt], norm_EE, norm_EI
```

```
# ----- CALCUL -----
```

```
# Exécution de la simulation
```

```
n_CN, n_LF, n_EE, n_EI = analyze_methods(psi0, V, dx, dt, Nt)
```

```
# Axe temporel
```

```
t_axis = np.linspace(0, Nt*dt, Nt)
```

```

# ----- FIGURE 1 -----

plt.figure(figsize=(10, 6)) # Taille figure

# Tracé des différentes méthodes
# avec l nomination des axes et les courbe de chaque methode
# le choix des couleurs juste pour distinguer entre les deferents courbes
plt.plot(t_axis, n_CN, label="Crank-Nicolson (Unitaire)", linewidth=2)
plt.plot(t_axis, n_LF, label="Leap-Frog")
plt.plot(t_axis, n_EI, label="Euler Implicite")
plt.plot(t_axis, n_EE, label="Euler Explicite")

plt.axhline(y=1.0, linestyle=':', label="Norme théorique = 1") # Référence
plt.ylim(0.8, 1.2)

plt.title("Conservation de la Norme")
plt.xlabel("Temps")
plt.ylabel("Norme")

plt.legend()
plt.grid()
plt.show()

# ----- FIGURE 2 -----

plt.figure(figsize=(10, 6))

# Erreur par rapport à 1 en échelle logarithmique
plt.semilogy(t_axis, np.abs(np.array(n_CN)-1), label="Erreur CN")
plt.semilogy(t_axis, np.abs(np.array(n_EI)-1), label="Erreur Euler Implicite")
plt.semilogy(t_axis, np.abs(np.array(n_LF)-1), label="Erreur Leap-Frog")
plt.semilogy(t_axis, np.abs(np.array(n_EE)-1), label="Erreur Euler Explicite")

plt.title("Erreur numérique (échelle log)")
plt.xlabel("Temps")
plt.ylabel("|Norme - 1|")

plt.legend()
plt.grid(True, which="both")
# afficher les resultats sur l'ordinateur
plt.show()

# cette partie est une suite de programme
# nous allons etudier l'effet de largeur et longueur de barriere de potentiel
# sur la fonction d'onde selon les 4 methodes
# et l'evalauton de l'energie dans le systeme avec 4 methodes

import numpy as np
import matplotlib.pyplot as plt
from scipy.sparse import diags
from scipy.sparse.linalg import spsolve

# =====
# PARAMÈTRES PHYSIQUES
# =====
hbar = 1.0
m = 1.0

```

```

# =====
# DISCRÉTISATION
# =====
L = 10.0
Nx = 400
dx = L/(Nx-1)
x = np.linspace(0, L, Nx)

dt = 0.001
Nt = 800

# =====
# CONDITION INITIALE (Paquet Gaussien)
# =====
x0 = 2.0
sigma = 0.4
k0 = 8.0

def initial_wave():
    psi = np.exp(-(x-x0)**2/(2*sigma**2)) * np.exp(1j*k0*x)
    psi /= np.sqrt(np.sum(np.abs(psi)**2)*dx)
    return psi

# =====
# FONCTION ENERGIE
# =====
def compute_energy(psi, V):
    d2psi = (np.roll(psi, -1) - 2*psi + np.roll(psi, 1))/dx**2
    T = -hbar**2/(2*m)*d2psi
    Hpsi = T + V*psi
    E = np.real(np.sum(np.conj(psi)*Hpsi)*dx)
    return E

# =====
# MÉTHODES NUMÉRIQUES
# =====

# ----- 1. CRANK-NICOLSON -----
def crank_nicolson(psi0, V):
    psi = psi0.copy()
    E_list = []

    H_diag = (hbar**2/(m*dx**2)) + V
    H_off = -hbar**2/(2*m*dx**2)*np.ones(Nx-1)

    H = diags([H_off, H_diag, H_off], [-1, 0, 1]).tocsc()
    I = diags([np.ones(Nx)], [0]).tocsc()

    A = I + 1j*dt/(2*hbar)*H
    B = I - 1j*dt/(2*hbar)*H

    for _ in range(Nt):
        psi = spsolve(A, B.dot(psi))
        E_list.append(compute_energy(psi, V))

    return psi, E_list

```

```

# ----- 2. EULER EXPLICITE -----
def euler_explicit(psi0, V):
    psi = psi0.copy()
    E_list = []

    for _ in range(Nt):
        d2psi = (np.roll(psi, -1) - 2*psi + np.roll(psi, 1))/dx**2
        Hpsi = -hbar**2/(2*m)*d2psi + V*psi
        psi = psi - 1j*dt/hbar*Hpsi
        E_list.append(compute_energy(psi, V))

    return psi, E_list

# ----- 3. EULER IMPLICITE -----
def euler_implicit(psi0, V):
    psi = psi0.copy()
    E_list = []

    alpha = 1j*dt*hbar/(2*m*dx**2)

    A = np.zeros((Nx, Nx), dtype=complex)
    for i in range(1, Nx-1):
        A[i, i-1] = -alpha
        A[i, i] = 1 + 2*alpha + 1j*dt*V[i]/(2*hbar)
        A[i, i+1] = -alpha
    A[0, 0] = A[-1, -1] = 1

    for _ in range(Nt):
        psi = np.linalg.solve(A, psi)
        E_list.append(compute_energy(psi, V))

    return psi, E_list

# ----- 4. LEAP-FROG -----
def leap_frog(psi0, V):
    psi_prev = psi0.copy()
    d2psi = (np.roll(psi_prev, -1) - 2*psi_prev + np.roll(psi_prev, 1))/dx**2
    Hpsi = -hbar**2/(2*m)*d2psi + V*psi_prev
    psi_curr = psi_prev - 1j*dt/hbar*Hpsi

    E_list = [compute_energy(psi_prev, V)]

    for _ in range(1, Nt):
        d2psi = (np.roll(psi_curr, -1) - 2*psi_curr + np.roll(psi_curr, 1))/dx**2
        Hpsi = -hbar**2/(2*m)*d2psi + V*psi_curr
        psi_next = psi_prev - 2j*dt/hbar*Hpsi
        psi_prev, psi_curr = psi_curr, psi_next
        E_list.append(compute_energy(psi_curr, V))

    return psi_curr, E_list

# =====
# POTENTIEL BARRIÈRE
# =====
def potential(width, height):
    V = np.zeros(Nx)

```

```

    center = Nx//2
    w = int(width/dx)
    V[center-w//2:center+w//2] = height
    return V

# =====
# SIMULATION PRINCIPALE
# =====
width = 0.5
height = 20
V = potential(width,height)

psi0 = initial_wave()

psi_CN, E_CN = crank_nicolson(psi0,V)
psi_EE, E_EE = euler_explicit(psi0,V)
psi_EI, E_EI = euler_implicit(psi0,V)
psi_LF, E_LF = leap_frog(psi0,V)

# =====
# 1. GRAPHIQUE ÉNERGIE
# =====
time = np.arange(Nt)*dt

plt.figure(figsize=(10,6))
plt.plot(time,E_CN,label="Crank-Nicolson")
plt.plot(time,E_LF,label="Leap-Frog")
plt.plot(time,E_EI,label="Euler Implicite")
plt.plot(time,E_EE,label="Euler Explicite")
plt.title("Évolution de l'Énergie Totale")
plt.xlabel("Temps")
plt.ylabel("Énergie")
plt.legend()
plt.grid()
plt.show()

# =====
# 2. COMPARAISON DENSITÉ FINALE
# =====
plt.figure(figsize=(10,6))
plt.plot(x,np.abs(psi_CN)**2,label="CN")
plt.plot(x,np.abs(psi_LF)**2,label="LF")
plt.plot(x,np.abs(psi_EI)**2,label="EI")
plt.plot(x,np.abs(psi_EE)**2,label="EE")
plt.plot(x,V/np.max(V)*np.max(np.abs(psi_CN)**2),'k--',label="Potentiel")
plt.legend()
plt.title("Comparaison des Méthodes")
plt.show()

# =====
# 3. EFFET LARGEUR POTENTIEL
# =====
widths = [0.2,0.5,1.0]
plt.figure(figsize=(10,6))

for w in widths:
    V = potential(w,height)
    psi_CN,_ = crank_nicolson(initial_wave(),V)

```

```

plt.plot(x,np.abs(psi_CN)**2, label=f"Largeur={w}")

plt.title("Effet de la Largeur du Potentiel (CN)")
plt.legend()
plt.show()

# =====
# 4. EFFET HAUTEUR POTENTIEL
# =====
heights = [5,15,30]
plt.figure(figsize=(10,6))

for h in heights:
    V = potential(width,h)
    psi_CN,_ = crank_nicolson(initial_wave(),V)
    plt.plot(x,np.abs(psi_CN)**2, label=f"Hauteur={h}")

plt.title("Effet de la Hauteur du Potentiel (CN)")
plt.legend()
plt.show()

```